Learnable Item Tokenization for Generative Recommendation

Wenjie Wang*
wenjiewang96@gmail.com
National University of Singapore
Singapore

Honghui Bao*
honghuibao2000@gmail.com
National University of Singapore
Singapore

Xinyu Lin
xylin1028@gmail.com
National University of Singapore
Singapore

Jizhi Zhang
cdzhangjizhi@mail.ustc.edu.cn
University of Science and Technology of China
Hefei, China

Yongqi Li
liyongqi0@gmail.com
The Hong Kong Polytechnic University
Hong Kong SAR, China

Fuli Feng†
fulifeng93@gmail.com
University of Science and Technology of China
Hefei, China

See-Kiong Ng
seekiong@nus.edu.sg
National University of Singapore
Singapore

Tat-Seng Chua
dcscts@nus.edu.sg
National University of Singapore
Singapore

ABSTRACT

Utilizing powerful Large Language Models (LLMs) for generative recommendation has attracted much attention. Nevertheless, a crucial challenge is transforming recommendation data into the language space of LLMs through effective item tokenization. Current approaches, such as ID, textual, and codebook-based identifiers, exhibit shortcomings in encoding semantic information, incorporating collaborative signals, or handling code assignment bias. To address these limitations, we propose LETTER (a LEarnable Tokenizer for generaTivE Recommendation), which integrates hierarchical semantics, collaborative signals, and code assignment diversity to satisfy the essential requirements of identifiers. LETTER incorporates Residual Quantized VAE for semantic regularization, a contrastive alignment loss for collaborative regularization, and a diversity loss to mitigate code assignment bias. We instantiate LETTER on two models and propose a ranking-guided generation loss to augment their ranking ability theoretically. Experiments on three datasets validate the superiority of LETTER, advancing the state-of-the-art in the field of LLM-based generative recommendation.

CCS CONCEPTS

• **Information systems** → **Recommender systems**.

KEYWORDS

Generative Recommendation, LLMs for Recommendation, Item Tokenization, Learnable Tokenizer

*denotes equal contribution. This research is supported by NExT Research Center.
†Corresponding author.

CIKM '24, October 21–25, 2024, Boise, ID, USA

ACM ISBN 979-8-4007-0436-9/24/10
https://doi.org/10.1145/3627673.3679569

ACM Reference Format:
Wenjie Wang, Honghui Bao, Xinyu Lin, Jizhi Zhang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Learnable Item Tokenization for Generative Recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM '24), October 21–25, 2024, Boise, ID, USA.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3627673.3679569

1 INTRODUCTION

Harnessing Large Language Models (LLMs) for generative recommendation has emerged as a prominent avenue, drawing substantial research interest [14, 32]. Leveraging the powerful abilities of LLMs (*e.g.,* rich world knowledge, reasoning, and generalization), numerous efforts are investigating LLMs for generative recommendation, showcasing the potential of building foundational LLMs to move beyond natural language processing to information retrieval and recommendation domains [6, 9]. As illustrated in Figure 1, given user historically interacted items, generative recommendation aims to utilize LLMs to generate target items as recommendations. A key problem to encoding and generating items via LLMs is *item tokenization*, which indexes each item via an identifier (*i.e.,* a token sequence), thereby bridging the gap between recommendation data and the language space of LLMs. As such, it is imperative to investigate item tokenization for LLM-based generative recommendation.

Existing item tokenization approaches have mainly explored three types of identifiers to index an item:

- ID identifiers assign each item with a unique numerical string (*e.g.,* "28,128"), ensuring the identifier uniqueness [9, 14]. However, numerical IDs are inefficient in encoding semantic information, making it challenging to generalize to cold-start items [38].
- Textual identifiers directly leverage item semantic information such as titles, attributes, or/and descriptions as item identifiers [1, 14]. Nevertheless, such textual identifiers suffer from the following issues: 1) semantic information is not hierarchically distributed from coarse to fine-grained in token sequence of the identifiers. This results in low generation probability if the first tokens of the target item are not closely aligned with user preference (*e.g.,* "With" and "Be" in movie titles); 2) textual identifiers

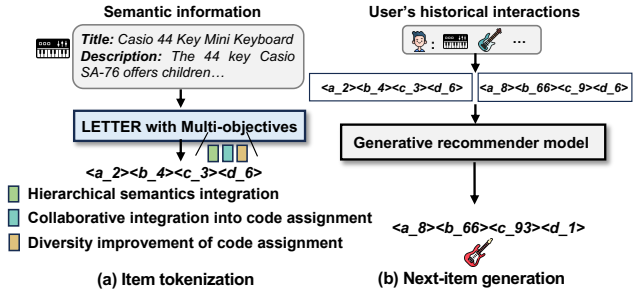

Figure 1: Overview of utilizing LETTER for LLM-based generative recommendation.

lack collaborative signals from user behaviors [1]. The token sequence of an identifier is solely determined by semantic information. As such, items with similar semantics yet differing collaborative signals exhibit similar token sequences, undermining the distinctiveness of items from the perspective of Collaborative Filtering (CF) (refer to Section 2 and Figure 2).

- Codebook-based methods adopt auto-encoders to encode item semantics into hierarchical code sequences as identifiers [32, 49]. However, similar to textual identifiers, codebook-based identifiers still lack collaborative signals in code sequences (see Section 2). Moreover, as shown in Figure 3, the assignment of codes to items is naturally imbalanced, causing the item generation bias where items with high-frequency codes are more readily generated.

In light of these considerations, an ideal identifier should meet the following criteria:

- Integrating hierarchical semantics into identifiers, where the token sequence initially encodes broad and coarse-grained semantics, progressively transitioning into more refined, fine-grained details. This aligns with the autoregressive generation characteristics of generative recommendation.
- Incorporating collaborative signals into the token assignment, which ensures that items with similar collaborative signals in user behaviors possess similar token sequences as identifiers.
- Improving the diversity of token assignments to alleviate the item generation bias, thereby ensuring fairness in item generation.

To this end, we propose a **LE**arnable **T**okenizer for genera**Tiv**E **R**ecommendation named LETTER. LETTER incorporates three kinds of regularization to enhance the codebook-based identifiers. In particular, semantic regularization first integrates Residual Quantized VAE (RQ-VAE) [16] to translate the item semantic information into the hierarchical identifiers [32]. Built on this, collaborative regularization utilizes a contrastive alignment loss to align semantic quantized embeddings in RQ-VAE with CF embeddings from a well-trained CF model (*e.g.,* LightGCN [11]). Moreover, LETTER introduces a diversity loss to enhance code embedding diversity to alleviate code assignment bias and item generation bias. We instantiate LETTER on two representative generative recommender models and propose a ranking-guided generation loss to boost the ranking ability of these generative models theoretically. Extensive experiments on three datasets with in-depth investigation have validated that LETTER can achieve superior item tokenization by simultaneously considering hierarchical semantics, collaborative signals, and code assignment diversity in

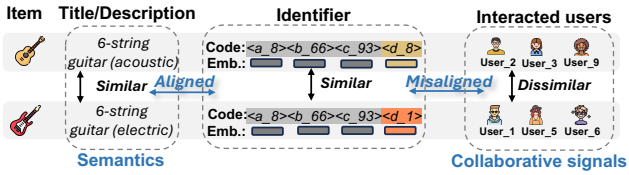

Figure 2: Misalignment between item identifiers and collaborative signals. "Emb." denotes "Embeddings".

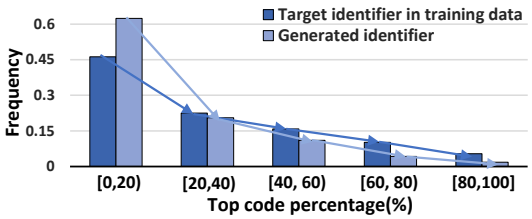

Figure 3: Illustration of code assignment bias and generation bias on Instruments.

identifiers. For reproducibility, we release our code and data at https://github.com/HonghuiBao2000/LETTER.

To sum up, the contributions of this work are as follows.

- We comprehensively analyze the necessary features of an ideal identifier. Accordingly, we propose a novel learnable tokenizer named LETTER, aimed at adaptively learning identifiers that encompass hierarchical semantics, collaborative signals, and code assignment diversity.
- We instantiate LETTER on two generative recommender models and leverage a ranking-guided generation loss to theoretically enhance the ranking ability of generative recommender models.
- We conduct extensive experiments on three datasets, coupled with the in-depth investigation with diverse settings, validating that LETTER outperforms existing item tokenization methods for generative recommendation.

2 ITEM TOKENIZATION

Generative recommendation aims to generate the next item given the user's historical interactions [17]. A crucial fundamental step in LLM-based generative recommendation lies in item tokenization, which assigns an identifier to each item. Each identifier is a sequence of tokens to assist LLMs with item encoding and generation. By item tokenization, LLM-based generative recommender models can 1) transform the user's historical interactions into a sequence of identifiers for item encoding and 2) autoregressively generate an item identifier for next-item recommendation (see Figure 1(b)).

Existing item token methods have explored ID identifiers [4, 9, 14, 42], textual identifiers [1, 46, 48], and codebook-based identifiers [32, 49]. However, ID identifiers have insufficient capability in semantic encoding [38]. Moreover, we present more details on the critical issues in textual and codebook-based identifiers.

- **Non-hierarchical semantics.** As mentioned in Section 1, semantic information is not hierarchically distributed from coarse to fine-grained in textual identifiers.
- **Lack of collaborative signals.** Previous textual and codebook-based identifiers do not consider collaborative signals in the composition of the token sequence. They typically inject collaborative signals into the token embeddings of identifiers during

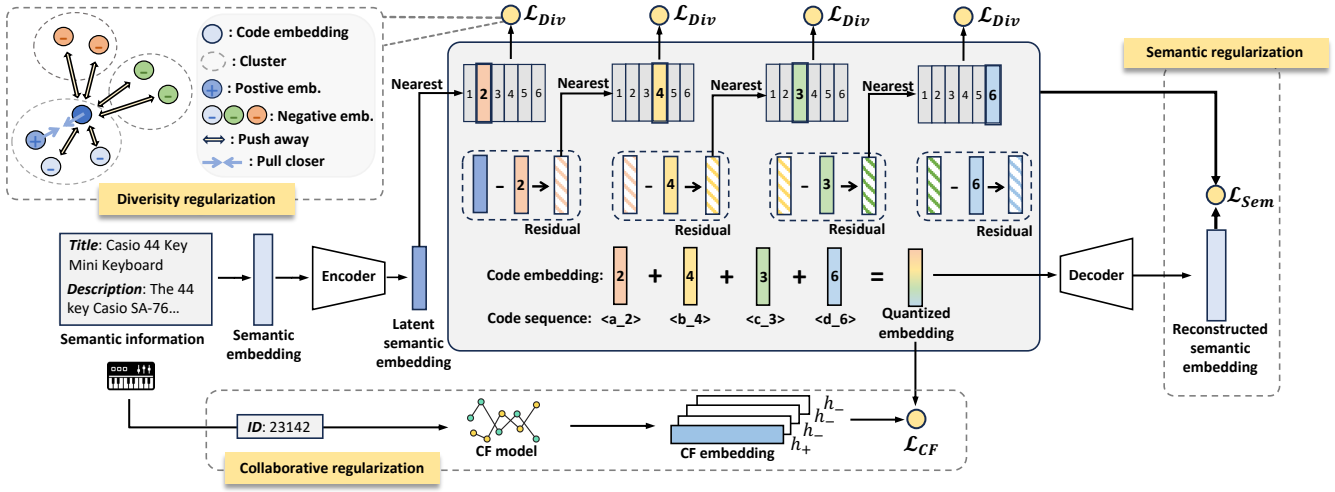

Figure 4: Illustration of LETTER with three kinds of regularization, where semantic regularization ensures the semantic encoding, collaborative regularization enhances the alignment between the identifiers' code sequence and collaborative signals, and diversity regularization alleviates the code assignment bias.

the LLM training stage (*e.g.,* TIGER [32] and LC-Rec [49]). However, as illustrated in Figure 2, two items with similar semantics always share similar token sequences and embeddings, making it challenging to align with collaborative signals. Injecting collaborative signals into identifier embeddings of these two items by training will cause collisions.

- **Code assignment bias.** Codebook-based methods additionally suffer from imbalanced code assignment, leading to item generation bias. We utilize TIGER [32] on Instruments dataset for empirical investigation. Specifically, we split item identifier codes into five groups based on their assignment popularity in the training data and obtain the generation frequency of TIGER. From Figure 3, we can observe that popular item identifiers are more likely to be generated, amplifying the generation bias.

In light of these, we consider three essential objectives to pursue an ideal identifier: 1) hierarchical semantic integration with the token sequence, 2) collaborative signals integration in the token sequence, and 3) high diversity of the code assignment.

3 METHOD

We elaborate on LETTER in Section 3.1 and apply LETTER to LLM-based generative recommender models in Section 3.2.

3.1 LETTER

To achieve the three objectives for item tokenization, we build LETTER for LLM-based generative recommendation. In particular, as depicted in Figure 4, LETTER can generate identifiers, *i.e.,* code sequences, with hierarchical semantics. Moreover, we introduce collaborative regularization to endow the code sequence with collaborative signals, and diversity regularization to alleviate the code assignment bias issue during tokenizer training.

- **Semantic regularization**. To pursue identifiers with hierarchical semantics, we develop LETTER based on RQ-VAE [16], which encodes the hierarchical item semantics into a code sequence. As a multi-level embedding quantizer, RQ-VAE recursively quantizes semantic residuals using a fixed number of codebooks, thus naturally achieving identifier with coarse to fine-grained semantics. As shown in Figure 4, the learnable tokenizer generates the hierarchical semantic identifier via two steps: 1) semantic embedding extraction, and 2) semantic embedding quantization.

Semantic embedding extraction. Given an item with its content information such as titles and descriptions, we first extract the semantic embedding $\boldsymbol{s}$ through a pre-trained semantic extractor, *e.g.,* LLaMA-7B [40]. The semantic embedding is then compressed into the latent semantic embedding $\boldsymbol{z} \in \mathbb{R}^d$ through an encoder $\boldsymbol{z} = \text{Encoder}(\boldsymbol{s})$.

Semantic embedding quantization. The latent semantic embedding $\boldsymbol{z}$ is then quantized into the code sequence through L-level codebooks, where L is the identifier length. Specifically, for each code level $l \in \{1, \ldots, L\}$, we have a codebook $C_l = \{\boldsymbol{e}_i\}_{i=1}^{N}$, where $\boldsymbol{e}_i \in \mathbb{R}^d$ is a learnable code embedding and N denotes the codebook size. Subsequently, the residual quantization can be formulated as

$$\begin{cases} c_l = \arg\min_i \|\boldsymbol{r}_{l-1} - \boldsymbol{e}_i\|^2, & \boldsymbol{e}_i \in C_l, \\ \boldsymbol{r}_l = \boldsymbol{r}_{l-1} - \boldsymbol{e}_{c_l}, \end{cases} \tag{1}$$

where c_l is the assigned code index from the l-th level codebook, $\boldsymbol{r}_{l-1}$ is the semantic residual from the last level, and we set $\boldsymbol{r}_0 = \boldsymbol{z}$. Intuitively, at each code level, LETTER finds the most similar code embedding with the semantic residual and assigns the item with the corresponding code index. After the recursive quantization, we eventually obtain the quantized identifier $\tilde{i} = [c_1, c_2, \ldots, c_L]$, and the quantized embedding $\hat{\boldsymbol{z}} = \sum_{l=1}^{L} \boldsymbol{e}_{c_l}$. The quantized embedding $\hat{\boldsymbol{z}}$ is then decoded to the reconstructed semantic embedding $\hat{\boldsymbol{s}}$. The loss for semantic regularization is formulated as:

$$\begin{cases} \mathcal{L}_{\text{Sem}} = \mathcal{L}_{\text{Recon}} + \mathcal{L}_{\text{RQ-VAE}}, & \text{where,} \\ \mathcal{L}_{\text{Recon}} = \|\boldsymbol{s} - \hat{\boldsymbol{s}}\|^2, \\ \mathcal{L}_{\text{RQ-VAE}} = \sum_{l=1}^{L} \|\text{sg}[\boldsymbol{r}_{l-1}] - \boldsymbol{e}_{c_l}\|^2 + \mu \|\boldsymbol{r}_{l-1} - \text{sg}[\boldsymbol{e}_{c_l}]\|^2, \end{cases} \tag{2}$$

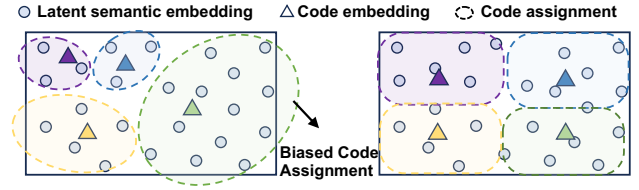

Figure 5: Illustration of the code assignment with biased distribution and uniform distribution of code embeddings.

where $\text{sg}[\cdot]$ is the stop-gradient operation [41], and μ is the coefficient to balance the strength between the optimization of code embeddings and encoder. Specifically, the reconstruction loss $\mathcal{L}_{\text{Recon}}$ aims to maintain the essential semantics information in the latent space such that the semantic embedding can be reconstructed from the quantized embedding. Besides, $\mathcal{L}_{\text{RQ-VAE}}$ reduces the residual error for all levels and jointly trains the encoder, and code embeddings [41]. By semantic regularization, the code sequence encodes hierarchical semantics, facilitating coarse-grained to fine-grained generation and cold-start generalization.

- **Collaborative regularization**. To inject collaborative signals into the code sequence instead of only code embeddings, we introduce collaborative regularization to align the quantized embedding $\hat{\boldsymbol{z}}$ and the CF embedding via contrastive learning. Specifically, we utilize a well-trained CF recommender model (*e.g.,* SASRec [15] and LightGCN [11]) to obtain CF embeddings of items, which is then aligned with the quantized embedding $\hat{\boldsymbol{z}}$ through a CF loss:

$$\mathcal{L}_{\text{CF}} = -\frac{1}{B}\sum_{i=1}^{B}\frac{\exp(<\hat{\boldsymbol{z}}_i, \boldsymbol{h}_i>)}{\sum_{j=1}^{B}\exp(<\hat{\boldsymbol{z}}_i, \boldsymbol{h}_j>)}, \tag{3}$$

where $\boldsymbol{h}$ refers to the CF embedding, $<\cdot,\cdot>$ denotes the inner product, and B is the batch size.

Intuitively, collaborative regularization encourages items with similar collaborative interactions to exhibit similar code sequences. In contrast to the typical codebook-based tokenization approaches such as TIGER [32] that relies solely on semantics to generate identifiers, we inject collaborative signals into the code embeddings by optimizing quantized embeddings, thereby altering the code assignment to better align with the collaborative patterns (refer to Section 4.3.2 for empirical evidence).

- **Diversity regularization**. To address the code generation bias, it is promising to learn a more uniform distribution of the code embeddings. As illustrated in Figure 5(a), a biased code embedding distribution is more likely to cause biased code assignment in the latent space, where some codes are assigned to more items. In contrast, a uniform code embedding distribution facilitates more balanced code assignment (Figure 5(b)). In light of these, we aim to improve the code embedding diversity for diverse code assignments.

Specifically, for each codebook, we cluster the code embeddings into K groups by constrained K-means [3]. We then regularize the clustered code embeddings by a diversity loss, which is defined as

$$\mathcal{L}_{\text{Div}} = -\frac{1}{B}\sum_{i=1}^{B}\frac{\exp(<\boldsymbol{e}^i_{c_l}, \boldsymbol{e}_+>)}{\sum_{j=1}^{N-1}\exp(<\boldsymbol{e}^i_{c_l}, \boldsymbol{e}_j>)}, \tag{4}$$

where $\boldsymbol{e}^i_{c_l}$ is the nearest code embedding of item i, $\boldsymbol{e}_+$ denotes the code embedding of a randomly selected sample from the same cluster of code c_l, and $\boldsymbol{e}_{j\in\{1,\ldots,N\}\setminus c_l}$ represents all code embeddings from the codebook except for $\boldsymbol{e}_{c_l}$. Intuitively, as shown in Figure 4, we pull the code embeddings from the same cluster closer and push the embeddings of codes from different clusters away from each other, thus encouraging diversity of the code embeddings and alleviating the code assignment bias issue (*cf.* Section 4.3.2 for empirical evidence).

- **Overall loss**. The training loss for LETTER is summarized as:

$$\mathcal{L}_{\text{LETTER}} = \mathcal{L}_{\text{Sem}} + \alpha\mathcal{L}_{\text{CF}} + \beta\mathcal{L}_{\text{Div}}, \tag{5}$$

where α and β are hyper-parameters to control the strength of collaborative regularization and diversity regularization, respectively.

3.2 Instantiation

To instantiate LETTER to LLM-based generative recommender models, we first train the tokenizer on the recommendation items via Eq. (5). And then the well-trained LETTER can be instantiated to tokenize items for the training and inference stages of LLMs.

3.2.1 ***Training.*** The training of generative recommender models includes item tokenization and model optimization phases. Item tokenization index each item into the identifier $\tilde{i} = [c_1, c_2, \ldots, c_L]$ via the well-trained LETTER. We then translate the user interaction sequences based on item identifiers. Formally, we have $\mathcal{D} = \{(x, y)\}$, where $x = [\tilde{i}_1, \tilde{i}_2, \ldots, \tilde{i}_M]$ denotes user's historically interacted items in chronological order, and $y = \tilde{i}_{M+1}$ refers to the identifier of user's next-interacted item.

- **Ranking-guided generation loss**. Existing work usually optimizes LLMs on $\mathcal{D} = \{(x, y)\}$ by a generation loss for negative log-likelihood minimization. Despite the effectiveness of the generation loss in various domains, such generation loss might overlook the ranking optimization over all items, thus undermining the recommendation performance. In this light, we propose a ranking-guided generation loss, which alters the temperature in the traditional generalization loss to emphasize the penalty for hard-negative samples, thereby enhancing the ranking ability of generative recommender models. Formally, the loss is defined as:

$$\begin{cases} \mathcal{L}_{\text{rank}} = -\sum_{t=1}^{|y|}\log P_\theta(y_t|y_{<t}, x), \\ P_\theta(y_t|y_{<t}, x) = \frac{\exp(p(y_t)/\tau)}{\sum_{v\in\mathcal{V}}\exp(p(v)/\tau)}, \end{cases} \tag{6}$$

where τ is the adjustable temperature to penalize the hard-negative samples, y_t refers to the t-th token of y, $y_{<t}$ represents the token sequence preceding y_t, θ denotes the learnable parameters of the generative model, and $\mathcal{V}$ is the token vocabulary[1].

Proposition 1. *For a given ranking-guided generation loss $\mathcal{L}_{rank}$ and a parameter τ, the following statements hold:*

- *Minimizing $\mathcal{L}_{rank}$ is equivalent to optimizing hard-negative items for users, where a smaller τ intensifies the penalty on hard negatives.*
- *The minimization of $\mathcal{L}_{rank}$ is associated with the optimization of one-way partial AUC [36], which is strongly correlated with*

[1]The token vocabulary is extended by adding the codes from LETTER.

ranking metrics such as Recall and NDCG, ultimately leading to an improvement in the top-K ranking ability.

The proof of this proposition is provided in Appendix 7.

3.2.2 ***Inference.*** To generate the next item, generative recommender models autoregressively generate the code sequence, which is formulated as $\hat{y}_t = \arg\max_{v \in \mathcal{V}} P_\theta(v|\hat{y}_{<t}, x)$. To ensure generating valid identifiers, we follow [14] to employ constrained generation [8] via Trie, which is a prefix tree [5] supporting the model to find all strictly valid successor tokens in autoregressive generation.

4 EXPERIMENTS

In this section, we conduct extensive experiments on three real-world datasets to answer the following research questions:

- **RQ1:** How does our proposed LETTER perform compared to different kinds of identifiers?
- **RQ2:** How do the components of LETTER (*e.g.,* collaborative regularization and diversity regularization) affect the performance?
- **RQ3:** How does LETTER perform under different settings (*i.e.,* identifier length, codebook size, strength of collaborative and diversity regularization, and temperature)?

4.1 Experimental Settings

4.1.1 ***Datasets.*** We use three real-world recommendation datasets from different domains: 1) **Instruments** is from the Amazon review datasets [29], which contains user interactions with rich music gears. 2) **Beauty** contains user interactions with extensive beauty products from Amazon review datasets [29]. 3) **Yelp**[2] is a popular dataset comprising business interactions on the Yelp platform. We adopt the pre-processing techniques from previous work [15, 32], discarding sparse users and items with interactions less than 5. We consider sequential recommendation setting[3] and use *leave-one-out* strategy [32, 49] to split datasets. For training, we follow [14, 49] to restrict the number of items in a user's history to 20.

4.1.2 ***Baselines.*** We compare LETTER with competitive baselines within two groups of work, traditional recommender models and LLM-based generative recommender models, with different types of identifiers (*i.e.,* ID, textual, and codebook-based identifiers): 1) **MF** [35] decomposes the user-item interactions into the user embeddings and the item embeddings in the latent space. 2) **Caser** [39] employs convolutional nerual networks to capture user's spatial and positional information. 3) **HGN** [28] utilizes graph neural networks to learn user and item representations for predicting user-item interactions. 4) **BERT4Rec** [37] leverages BERT's pre-trained language representations to capture semantic user-item relationships. 5) **LightGCN** [11] is a lightweight graph convolutional network model focusing on high-order connections between users and items. 6) **SASRec** [15] employs self-attention mechanisms to capture long-term dependencies in user interaction history. 7) **BIGRec** [1] is an LLM-based generative recommender model using textual identifiers, with each item represented by its title. 8) **P5-TID** [14] uses items' title as textual identifiers for LLM-based generative recommender model. 9) **P5-SemID** [14] leverages item metadata (*e.g.,* attributes) to construct ID identifiers for LLM-based generative recommender model. 10) **P5-CID** [14] incorporates collaborative signals into the identifier for LLM-based generative recommender models through a spectral clustering tree derived from item co-appearance graphs. 11) **TIGER** [32] introduces codebook-based identifiers via RQ-VAE, which quantizes semantic information into code sequence for LLM-based generative recommendation. 12) **LC-Rec** [49] uses codebook-based identifiers and auxiliary alignment tasks to better utilize knowledge in LLMs by connecting generated code sequences with natural language.

- **Evaluation settings.** We use two metrics: top-K Recall (R@K) and NDCG (N@K) with K = 5 and 10, following [32].

4.1.3 ***Implementation Details.*** We instantiate LETTER on two representative LLM-based generative recommender models, *i.e.,* TIGER [32] and LC-Rec [49]. As for TIGER, since the official implementations have not been released, we follow the paper for implementation. For LC-Rec, we perform the parameter-efficient fine-tuning technique LoRA [12] to fine-tune LLaMA-7B [40]. To obtain item semantic embedding, we follow [49] to adopt LLaMA-7B for encoding item content information. We obtain 32-dimensional item embedding from SASRec [15] for collaborative regularization and set cluster number K at 10 for diversity regularization. All the experiments are conducted on 4 NVIDIA RTX A5000 GPUs.

For item tokenization, we use 4-level codebooks for RQ-VAE, where each codebook comprises 256 code embeddings with a dimension of 32. We train LETTER for 20k epochs via AdamW [27] optimizer with a learning rate of 1e-3 and a batch size of 1,024. We follow [32] to set μ as 0.25, and search α in a range of {1e-1, 2e-2, 1e-2,1e-3}, β in a range of {1e-2, 1e-3, 1e-4,1e-5}. After LETTER training, we fine-tune TIGER and LC-Rec for convergence based on the validation performance with a learning rate in {1e-3, 5e-4} and {1e-4, 2e-4, 3e-4}.

4.2 Overall Performance (RQ1)

We evaluate LETTER on two SOTA LLM-based generative recommender models. The performances of LETTER instantiated on TIGER (LETTER-TIGER) and LC-REC (LETTER-LC-Rec) are reported in Table 1, from which we have the following observations:

- Among the LLM-based models with ID identifiers (P5-CID and P5-SemID), P5-CID usually yields better performance than P5-SemID over the three datasets. This is because P5-SemID assigns item identifiers based on the item categories, *e.g.,* "string" and "keyboard" for instrument products, which might 1) fail to capture fine-grained semantic information and 2) hinder the learning of collaborative behavior in LLMs' fine-tuning stage due to the misalignment between semantic and collaborative signals (*cf.* Figure 2 in Section 2). In contrast, P5-CID leverages collaborative signals from the item co-appearance graph to assign identifiers, which is beneficial for LLM-based recommender models to capture collaborative patterns in user behaviors.
- Among all LLM-based baselines with different kinds of identifiers, models with codebook-based identifiers (TIGER and LC-Rec) outperform models with ID identifiers (P5-SemID and P5-CID) and that with textual identifiers (BIGRec and P5-TID) in most cases. This is attributed to the incorporation of hierarchical

[2]https://www.yelp.com/dataset.
[3]We focus on sequential recommendation setting due to its high practicality and significant potential in real-world applications.

Table 1: Overall performance comparison between the baselines and LETTER instantiated on two competitive LLM-based generative recommender models on three datasets. The bold results highlight the better performance in comparing the backend models with and without LETTER.

Model	Instruments				Beauty				Yelp			
	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10
MF	0.0479	0.0735	0.0330	0.0412	0.0294	0.0474	0.0145	0.0191	0.0220	0.0381	0.0138	0.0190
Caser	0.0543	0.0710	0.0355	0.0409	0.0205	0.0347	0.0131	0.0176	0.0150	0.0263	0.0099	0.0134
HGN	0.0813	0.1048	0.0668	0.0774	0.0325	0.0512	0.0206	0.0266	0.0186	0.0326	0.0115	0.0159
Bert4Rec	0.0671	0.0822	0.0560	0.0608	0.0203	0.0347	0.0124	0.0170	0.0186	0.0291	0.0115	0.0159
LightGCN	0.0794	0.0100	0.0662	0.0728	0.0305	0.0511	0.0194	0.0260	0.0248	0.0407	0.0156	0.0207
SASRec	0.0751	0.0947	0.0627	0.0690	0.0380	0.0588	0.0246	0.0313	0.0183	0.0296	0.0116	0.0152
BigRec	0.0513	0.0576	0.0470	0.0491	0.0243	0.0299	0.0181	0.0198	0.0154	0.0169	0.0137	0.0142
P5-TID	0.0000	0.0001	0.0000	0.0000	0.0182	0.0432	0.0132	0.0254	0.0184	0.0263	0.0130	0.0155
P5-SemID	0.0775	0.0964	0.0669	0.0730	0.0393	0.0584	0.0273	0.0335	0.0202	0.0324	0.0131	0.0170
P5-CID	0.0809	0.0987	0.0695	0.0751	0.0404	0.0597	0.0284	0.0347	0.0219	0.0347	0.0140	0.0181
TIGER	0.0870	0.1058	0.0737	0.0797	0.0395	0.0610	0.0262	0.0331	0.0253	0.0407	0.0164	0.0213
LETTER-TIGER	**0.0909**	**0.1122**	**0.0763**	**0.0831**	**0.0431**	**0.0672**	**0.0286**	**0.0364**	**0.0277**	**0.0426**	**0.0184**	**0.0231**
LC-Rec	0.0824	0.1006	0.0712	0.0772	0.0443	0.0642	0.0311	0.0374	0.0230	0.0359	0.0158	0.0199
LETTER-LC-Rec	**0.0913**	**0.1115**	**0.0789**	**0.0854**	**0.0505**	**0.0703**	**0.0355**	**0.0418**	**0.0255**	**0.0393**	**0.0168**	**0.0211**

semantics with different granularities via RQ-VAE, which distinguishes between similar items more effectively by grasping fine-grained details. Meanwhile, the inferior performance of BIGRec and P5-TID with textual identifiers is probably due to the potential misalignment between items' similar semantics and dissimilar interactions, thus hurting the learning of collaborative signals for accurate recommendation.

- LETTER consistently exhibits significant improvements for the backend models (TIGER and LC-Rec) across three datasets, validating the effectiveness of our method. The superior performance is attributed to: 1) the CF integration during code assignment, which aligns the collaborative signals and the semantic embeddings, thereby encouraging items with similar interactions or semantics to have similar code sequence and addressing the misalignment issue between semantics and collaborative signals (refer to Section 4.3.4 for further analysis of CF integration). 2) The improved diversity of token assignments, which is achieved by regularizing the representation space of code embeddings, thereby alleviating item generation bias caused by code assignment bias (refer to Section 4.3.2 for detailed analysis).

4.3 In-depth Analysis

4.3.1 ***Ablation Study (RQ2).*** To thoroughly investigate the effect of each regularization, we compare the following five variants of LETTER on TIGER: (0) we solely employ semantic regularization for item tokenization, which is equivalent to TIGER. (1) We incorporate both semantic and collaborative regularization for item tokenization, denoted as "TIGER w/ c. r.". (2) We involve both semantic and diversity regularization for item tokenization, referred to as "TIGER w/ d. r.". (3) We utilize semantic, collaborative, and diversity regularization for item tokenization and train TIGER with original generation loss, denoted as "(1) w/ d. r.". (4) We employ all regularizations for item tokenization and apply ranking-guided generation loss, *i.e.,* LETTER-TIGER.

Results on Instruments and Beauty are depicted in Table 2. We omit the results on Yelp with similar observations to save space.

Table 2: Ablation study of LETTER-TIGER.

Variants	Instruments		Beauty	
	R@10	N@10	R@10	N@10
(0): TIGER	0.1058	0.0797	0.0610	0.0331
(1): TIGER w/ c. r.	0.1078	0.0810	0.0660	0.0351
(2): TIGER w/ d. r.	0.1075	0.0809	0.0618	0.0335
(3): (1) w/ d. r.	0.1092	0.0819	0.0672	0.0357
(4): LETTER-TIGER	**0.1122**	**0.0831**	**0.0672**	**0.0364**

From the table, we have several key observations: 1) incorporating either collaborative or diversity regularization can improve the performance of TIGER, which validates the effectiveness of injecting collaborative signals into code assignment and improving the diversity of code embeddings, respectively. 2) Optimizing item tokenization with all three regularizations will further improve the performance (superior performance of (3) than (0-2)), indicating the effectiveness of considering both semantics and collaborative information in the code assignment with improved diversity. 3) LETTER-TIGER achieves the best performance, which leverages ranking-guided generation loss. This shows the effectiveness of suppressing hard-negative samples by altering temperature to strengthen the ranking ability.

4.3.2 ***Code Assignment Distribution (RQ2).*** We further investigate whether diversity regularization can effectively mitigate the code assignment bias in item tokenization. We compare the distributions of the first code of identifier tokenized by 1) TIGER with (w/) and without diversity regularization (left figure in Figure 6), and 2) TIGER equipped with collaborative regularization with and without diversity regularization (right figure in Figure 6), respectively. Specifically, for each tokenization approach, we first tokenize items with the well-trained tokenizer. We then sort the first code of the identifier according to the frequency and divide the first codes into groups with each of the groups containing 15 codes.

From Figure 6, we observe that: 1) incorporating diversity regularization effectively promotes a more uniform distribution of the first code, thus alleviating code assignment bias and potentially mitigating the item generation bias of generative recommender models.

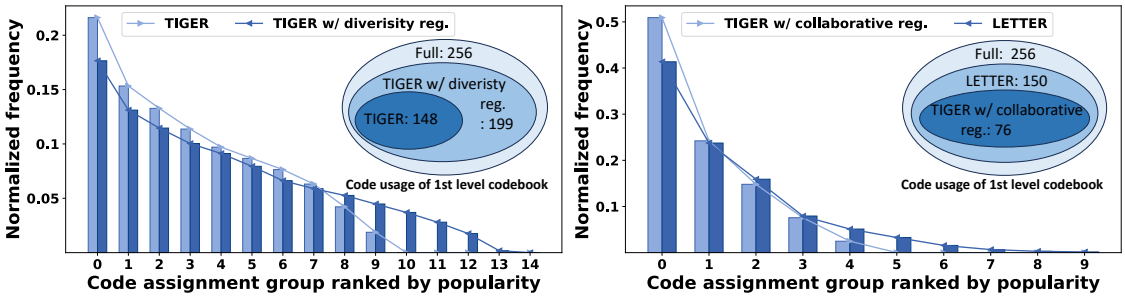

Figure 6: Illustration of smoother code assignment distribution on Instruments. The left figure compares the code assignment between TIGER and TIGER with diversity regularization and the right figure compares between TIGER with collaborative regularization and LETTER. "reg" denotes "regularization".

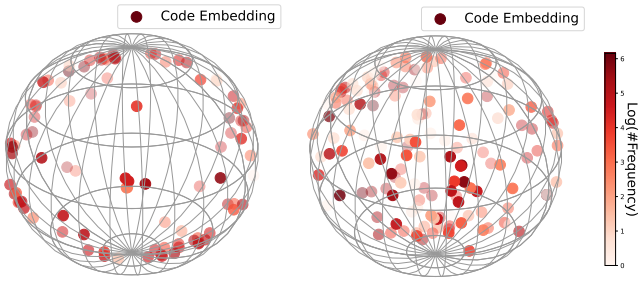

Figure 7: Code embedding distribution of LETTER with and without diversity regularization on Instruments.

2) The incorporation of diversity regularization significantly raises the utilization rate of codes in the first-level codebook, thereby improving diversity in code assignment. 3) Although adding collaborative regularization decreases the code utilization of TIGER (from 148 to 76), integrating diversity regularization inversely compensates for the drop in code utilization. As such, LETTER can capture collaborative signals and maintain a high code utilization, simultaneously meeting the multiple criteria of an ideal identifier.

4.3.3 ***Code Embedding Distribution (RQ2).*** To analyze whether diversity regularization can mitigate the biased distribution of code embeddings, we visualize the code embeddings of LETTER and LETTER without (w/o) diversity regularization. we first obtain the code embeddings from the first-level codebook and then perform PCA to reduce the high-dimensional embeddings into the 3-dimensional space for visualization. As shown in Figure 7, we present the 3-dimensional code embeddings in a spherical plot, where each point represents a code embedding vector, and darker colors indicate that the code is assigned to more items. By comparing (a) LETTER w/o diversity regularization and (b) LETTER, we can find that the first code embeddings of LETTER are more evenly distributed in the embedding representation space compared to LETTER w/o diversity regularization. This validates the effectiveness of diversity regularization to achieve a more diverse distribution of code embeddings in the representation space, fundamentally alleviating the problem of biased code assignment in Figure 5(a).

4.3.4 ***Investigation on Collaborative Signals in Identifiers (RQ2).*** To verify whether LETTER encodes collaborative signals into identifiers as expected, we design two experiments for analysis:

Table 3: Ranking performance with quantized embeddings.

Dataset	Model	R@5	R@10	N@5	N@10
Instruments	TIGER	0.0050	0.0150	0.0024	0.0049
	LETTER	**0.0080**	**0.0159**	**0.0038**	**0.0058**
Beauty	TIGER	0.0128	0.0213	0.0064	0.0085
	LETTER	**0.0175**	**0.0343**	**0.0076**	**0.0118**

Table 4: Code similarity between items with similar collaborative signals.

	Instruments	Beauty
TIGER	0.0849	0.1135
LETTER	**0.2760**	**0.3312**

- **Ranking experiment**. We aim to assess the ranking performance of LETTER by utilizing the quantized embeddings of items for interaction prediction. Specifically, we first obtain the item's quantized embedding $\hat{z}$ from the well-trained tokenizer. We then replace the item embeddings of the well-trained traditional CF model (*i.e.,* SASRec) with the quantized embeddings for interaction prediction. Intuitively, identifiers that effectively capture collaborative signals will lead to a better ranking performance. From Table 3, we can observe that LETTER outperforms TIGER by a large margin, indicating the effective incorporation of collaborative signals.
- **Similarity experiment**. Our target is to verify whether the items with similar collaborative signals exhibit similar identifiers. We first pair every item with its most similar item based on the similarity from pre-trained CF embeddings. Next, we assess the similarity of the code sequence between the two items within the item pairs. Specifically, we measure the overlap degree between the code sequences of the two items and report the averaged results of all items in Table 4. We can find that LETTER achieves more similar code sequences for the items with similar collaborative signals than TIGER, thereby alleviating the misalignment issue between semantics and the collaborative similarity.

4.3.5 ***Hyper-Parameter Analysis (RQ3).*** We further investigate several important hyper-parameters introduced in LETTER to facilitate future applications.

- **Identifier length** L**.** We vary L from 2 to 8 and present the results in Figure 8. It is observed that 1) the performance improves as we increase L from 2 to 4. Because overly short identifiers may lose some fine-grained information, resulting in weaker expressiveness of idetentifiers. 2) Continuously increasing the identifier length from 4 to 8 will inversely degrade the performance. This is reasonable because autoregressive generation suffers from error

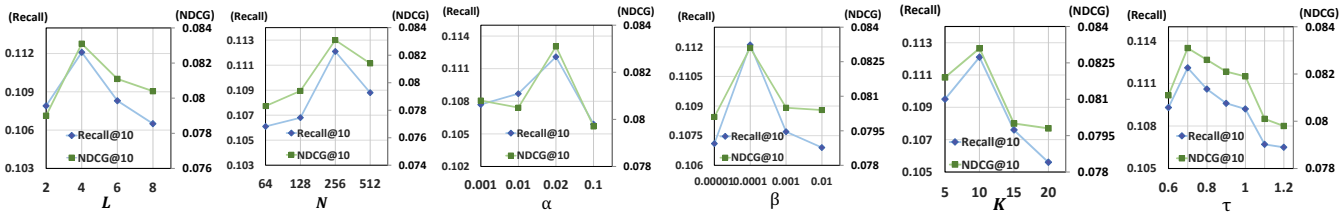

Figure 8: Performance of LETTER-TIGER over different hyper-parameters on Instruments.

accumulation [34]. Since accurate item generation requires the correctness of all generated codes in the sequence, generating longer identifiers accurately is more challenging than shorter identifiers.

- **Codebook size N.** We evaluate LETTER with different codebook sizes $N = 64$, 128, 256, and 512. Results in Figure 8 show that 1) Gradually increasing N tends to perform better. The worse performance of small-sized codebooks may be caused by the limited diversity of code selection, thus failing to distinguish items effectively. However, 2) blindly expanding N might inversely hurt the performance, where larger codebook size might be more susceptible to noise in the item's semantic information, potentially leading to the over-fitting of some meaningless semantics.
- **Strength of collaborative regularization α.** We vary α from 0.001 to 0.1 and present the results in Figure 8. We can find that 1) as α continuously increases, the overall trend of performance is observed to be generally improved. This is reasonable because larger α implies a stronger injection of collaborative patterns, but an overly large α may instead interfere with semantic regularization. 2) Empirically, we recommend setting $\alpha = 0.02$ because it probably achieves a proper balance between semantic and collaborative regularization, thus leading to the best performance.
- **Strength of diversity regularization β.** We examine the influence of different diversity regularization strengths by adjusting β from 0.00001 to 0.01. The results are in Figure 8 and we observe that applying diversity regularization with a slight strength is sufficient to enhance the diversity of code assignment (significant improvements from $\beta = 0.00001$ to $\beta = 0.0001$). In contrast, an excessive amount of diversity signal may interfere with the tokenizer's integration of semantic and collaborative signals.
- **Cluster K.** Based on the results with various values of K ranging from 5 to 20, as illustrated in Figure 8, we can observe that decreasing or increasing K from 10, the performance tends to decline. This is understandable because overly large clusters contain too many code embeddings, causing embeddings within the same cluster to be insufficiently close, whereas overly small clusters have relatively few code embeddings, leading to embeddings within the same cluster being overly close.
- **Temperature τ.** From the results of LETTER-TIGER with multiple values of τ ranging from 0.6 to 1.2 as shown in Figure 8, we can find that decreasing τ from 1.2 to 0.7, the performance tends to decline. This is reasonable as decreasing temperature casts more emphasis on the penalty for hard negatives, where the ranking ability is strengthened, thus improving the recommendation performance. Nevertheless, we should carefully choose τ since a too-small τ might suppress the possibility of hard negative samples being considered as positive samples for other users.

5 RELATED WORK

- **LLMs for Generative Recommendation.** LLMs have shown promising prospects for generative recommendation [10, 20, 24, 25], where item tokenization is a crucial problem. Existing work on item tokenization primarily investigate three types of identifiers for item indexing: ID identifiers [4, 9, 14, 42], textual identifiers [1, 7, 19, 21, 46, 48], codebook-based identifiers [32, 42, 49]. In early stages, randomly assigned IDs struggled to effectively encode semantic information and collaborative patterns of items. So ID identifiers such as SemID [14] and CID [14] use item semantic information and collaborative signals, respectively, to build tree-like structures for generating identifiers. However, such a fixed and unlearnable structure is hard to efficiently and effectively represent item similarity and adapt new items. Alternatively, textual identifiers leverage detailed descriptive information of items as identifiers. Nevertheless, these methods struggle to hierarchically encode semantic information or integrate collaborative signals. Codebook-based identifiers leverage codebooks and attempt to integrate semantic information and collaborative signals during training process. However, they overlook the misalignment introduced by incorporating collaborative signals into fixed code sequences, as well as the code assignment bias. Specifically, LC-Rec uses the Sinkhorn-Knopp Algorithm to fairly consider intra-layer codes through iterative normalization, missing the essence of code assignment: the distribution of code embeddings. In this work, we propose a superior identifier that integrates hierarchical semantic with collaborative signals and code assignment diversity.
- **LLMs for Discriminative Recommendation.** LLMs are used not only for generating recommendation but also extensively in different discriminative recommendation tasks. Two main approaches are employed in utilizing LLMs for discriminative recommendation. 1) LLM-enhanced traditional recommender. Many studies leverage LLMs for data augmentation [26, 43, 45] and representation [31, 33, 44]. 2) LLM-based recommender, which utilizes LLMs as recommender models directly. Some methods [18, 22, 47] incorporate a projection layer to calculate the matching score between users and items for ranking. Besides, another line [2, 23, 30] leverages LLMs for click-through rate (CTR) prediction, combining user and item features to determine whether the user will like this item. In this work, we mainly focus on the identifier design for generative recommendation, and thus we do not delve into in-depth discussion and comparison with discriminative methods.

6 CONCLUSION AND FUTURE WORK

In this study, we undertook a thorough analysis of the optimal features required for item tokenization in generative recommendation. Subsequently, we introduced LETTER, a learnable tokenizer

incorporating three forms of regularization aimed at capturing hierarchical semantics, collaborative signals, and code assignment diversity within item identifiers. We applied LETTER to two prominent generative recommender models and employed a ranking-guided generation loss to enhance their ranking performance. Extensive experiments substantiate the superiority of LETTER in item tokenization for generative recommendation models.

This work highlights the importance of item tokenization on generative recommendation, leaving some promising direction for future exploration. 1) Tokenization with rich user behaviors to enable generative recommender models to infer user preference from multiple user actions. 2) LETTER has the potential to tokenize cross-domain items for open-ended recommendation, enabling generative recommender models to leverage multi-domain user behaviors and items for user preference reasoning and next-item recommendation. 3) It is promising to combine user instructions in natural language with user interaction history tokenized by LETTER for personalized recommendations, achieving collaborative reasoning with complex natural language instructions and item tokens in the space of generative recommender models.

7 APPENDIX

Our proof is primarily divided into hard negative mining and relation with ranking two parts.

- **Hard Negative Mining.** This subsection primarily proofs two objects: Firstly, the ranking-guided loss $\mathcal{L}_{Rank}$ can mine hard negative samples. Secondly, as the value of τ decreases, there is an increased focus on hard negative samples. We first prove hard negative mining ability of $\mathcal{L}_{Rank}$:

$$\nabla \mathcal{L}_{rank} = -\nabla \sum_{t=1}^{|y|} log P_\theta(y_t|y_{<t}, x) = -\sum_{t=1}^{|y|} \nabla log \frac{\exp(p(y_t)/\tau)}{\sum_{v\in\mathcal{V}} \exp(p(v)/\tau)}. \tag{7}$$

We shall demonstrate that when t-1 tokens are determined, our loss is associated with hard negative mining on the t-th token.

$$-\nabla log \frac{\exp(p(y_t)/\tau)}{\sum_{v\in\mathcal{V}} \exp(p(v)/\tau)} = -\frac{1}{\tau}\nabla p(y_t) + \nabla log \sum_{v\in\mathcal{V}} \exp(p(v)/\tau) \tag{8}$$

$$= \frac{\sum\limits_{v\in\mathcal{V}, v\neq p_t} \exp(p(v)/\tau)}{\tau \sum\limits_{v\in\mathcal{V}} \exp(\frac{p(v)}{\tau})} [\sum_{\substack{v\in\mathcal{V},\\ v\neq p_t}} \left(\frac{\exp(\frac{p(v)}{\tau})}{\sum_{\substack{v\in\mathcal{V},\\ v\neq p_t}} \exp(\frac{p(v)}{\tau})} \nabla p(v) \right) - \nabla p(y_t)]. \tag{9}$$

Next, we analyze the gradient $p(v)$ that associated with hard negative mining. Noting that this gradient is equivalent to minimizing the hard negative mining loss $\mathcal{L}_H := \sum_{v\in\mathcal{V}, v\neq p_t} w(p(v),\tau)p(v)$, where $w(p(v),\tau) := sg\left[\frac{\exp(p(v)/\tau)}{\sum_{v\in\mathcal{V}, v\neq p_t}\exp(p(v)/\tau)}\right]$, $sg[\cdot]$ denotes stop-gradient operation. We can observe that: $w(p(v),\tau) \propto p(v)$ This means the harder the negative sample is, the larger $w(p(v),\tau)$ is, demonstrating the ability of hard negative sample mining of our ranking-guided loss $\mathcal{L}_{Rank}$.

Next, we elucidate that with the diminishing value of τ, there is a heightened emphasis on hard negative samples. For a given negative sample v', if $p(v') > \frac{\sum_{v\in\mathcal{V}, v\neq p_t}\exp(p(v)/\tau)p(v)}{\sum_{v\in\mathcal{V}, v\neq p_t}\exp(p(v)/\tau)}$, we have $\partial_\tau w(p(v'),\tau) < 0$, showing adjusting the value of τ in the ranking-guided loss mechanism increases the weights of more difficult negative samples with higher $p(v)$ and decreases the weights of simpler negative samples.

- **Relation with Ranking.** Subsequently, we reveal the correlation between our loss and ranking metric. Firstly, let us observe the gradient in Eq.9, we have:

$$\begin{aligned} &\sum_{v\in\mathcal{V}, v\neq p_t} \left(\frac{\exp(p(v)\tau)}{\sum_{\substack{v\in\mathcal{V},\\ v\neq p_t}} \exp(p(v)/\tau)} \nabla p(v) \right) - \nabla p(y_t) \\ &= \nabla \underbrace{\left\{ \tau \log \left[\sum_{v\in\mathcal{V}, v\neq p_t} \exp(L_\theta(v, y_t)/\tau) \right] + \tau\rho \right\}}_{\mathcal{L}^{\rho,\tau}_{\text{DRO}}}, \end{aligned} \tag{10}$$

where we define: $L_\theta(v, y_t) := p(v) - p(y_t)$, θ is the notation for the parameters in the recommender system, Eq. 10 for $\forall\rho \in \mathcal{R}$ established. This means $\mathcal{L}_{rank}$ is a surrogate loss for $\mathcal{L}^{\rho,\tau}_{\text{DRO}}$. Then we analyze $\mathcal{L}^{\rho,\tau}_{\text{DRO}}$:

$$\mathcal{L}^{\rho,\tau}_{\text{DRO}} \geq \underbrace{\inf_{\tau>0} \left\{ \tau \log \left[\sum_{v\in\mathcal{V}, v\neq p_t} \exp(L_\theta(v, y_t)/\tau) \right] + \tau\rho \right\}}_{\mathcal{L}^{\rho}_{DRO}}, \tag{11}$$

which demonstrates that $\mathcal{L}^{\rho,\tau}_{\text{DRO}}$ is a upper bound for every $\mathcal{L}^{\rho}_{DRO}$. Similar to [13], minimizing $\mathcal{L}^{\rho}_{DRO}$ is the close form of the following optimization problem:

$$\min_\theta \max_Q E_Q\left[L_\theta(v, y_t)\right], \text{Subject to } D_{KL}(Q||P_0) \leq \rho, \tag{12}$$

where P_0 is the uniform distribution of negative samples, D_{KL} is the KL divergence.

Lemma 1. *(Brought from Theorem 2 in [36]) optimize problem 12 is associated with optimizing one-way partial AUC (OPAUC). Furthermore, $\mathcal{L}^{\rho}_{DRO}$ is a surrogate version of OPAUC($e^{-\rho}$) object. The formation of OPAUC(β), $\beta \in [0, 1]$ can be presented as following [50]:*

$$OPAUC(\beta) = \int_0^\beta TPR[FPR^{-1}(s)]ds, \tag{13}$$

where TPR is true positive rate and FPR is false positive rate.

Subsequently, we shall demonstrate the relation between $OPAUC(\beta)$ and top-K ranking metric, finally showing the relation between ranking-guided loss and ranking metrics:

Theorem 1. *Considering only one positive sample, OPAUC has strong correlation with top-K ranking metric:*

$$\begin{aligned} &Recall@K = \mathbb{I}(OPAUC(K/(|\mathcal{V}|-1)) > 0), \\ &NDCG@K = \frac{\mathbb{I}(OPAUC(K/(|\mathcal{V}|-1)) > 0)}{\log_2((K-1)(1-OPAUC(K/(|\mathcal{V}|-1))) + 2)}, \end{aligned} \tag{14}$$

where $\mathbb{I}(\cdot)$ is the indicator function, for $\forall K > 1$ exists.

Proof. Since we only have one positive samples, we can derive the exact rank position of the positive sample based on OPAUC. We use r_p to denote the ranking position of the positive sample. Set $\beta = K/(|V|-1)$, we have: $OPAUC(K/(|V|-1)) = (K-r)/(K-1)\mathbb{I}(r_p < K)$. If $r_p < K$, we have: $r_p = (K-1)(1-OPAUC(K/(|V|-1)))$. At the same time, the value of $OPAUC(K/(|V|-1))$ can also be used to determine whether $r_p < K$: $\mathbb{I}(OPAUC(K/(|V|-1) > 0) = \mathbb{I}(r_p < K)$. Since we have: $Recall@K = \mathbb{I}(r_p < K), NDCG@K = \mathbb{I}(r_p < K)/\log_2(r_p+1)$ substitute the results of r_p in to $Recall@K$ and $NDCG@K$ to finish the proof. □

REFERENCES

[1] Keqin Bao, Jizhi Zhang, Wenjie Wang, Yang Zhang, Zhengyi Yang, Yancheng Luo, Fuli Feng, Xiangnaan He, and Qi Tian. A bi-step grounding paradigm for large language models in recommendation systems. *arXiv:2308.08434*, 2023.

[2] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *RecSys*. ACM, 2023.

[3] Paul S Bradley, Kristin P Bennett, and Ayhan Demiriz. Constrained k-means clustering. *Microsoft Research, Redmond*, 20(0):0, 2000.

[4] Zhixuan Chu, Hongyan Hao, Xin Ouyang, Simeng Wang, Yan Wang, Yue Shen, Jinjie Gu, Qing Cui, Longfei Li, Siqiao Xue, et al. Leveraging large language models for pre-trained recommender systems. *arXiv preprint arXiv:2308.10837*, 2023.

[5] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to algorithms*. MIT press, 2022.

[6] Zeyu Cui, Jianxin Ma, Chang Zhou, Jingren Zhou, and Hongxia Yang. M6-rec: Generative pretrained language models are open-ended recommender systems. In *KDD*. ACM, 2022.

[7] Sunhao Dai, Ninglu Shao, Haiyuan Zhao, Weijie Yu, Zihua Si, Chen Xu, Zhongxiang Sun, Xiao Zhang, and Jun Xu. Uncovering chatgpt's capabilities in recommender systems. In *RecSys*, pages 1126–1132. ACM, 2023.

[8] N De Cao, G Izacard, S Riedel, and F Petroni. Autoregressive entity retrieval. In *ICLR*, volume 2021. ICLR, 2020.

[9] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *RecSys*, pages 299–315. ACM, 2022.

[10] Yuqi Gong, Xichen Ding, Yehui Su, Kaiming Shen, Zhongyi Liu, and Guannan Zhang. An unified search and recommendation foundation model for cold-start scenario. In *CIKM*, pages 4595–4601, 2023.

[11] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *SIGIR*, pages 639–648. ACM, 2020.

[12] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv:2106.09685*, 2021.

[13] Zhaolin Hu and L Jeff Hong. Kullback-leibler divergence constrained distributionally robust optimization. *Available at Optimization Online*, 1(2):9, 2013.

[14] Wenyue Hua, Shuyuan Xu, Yingqiang Ge, and Yongfeng Zhang. How to index item ids for recommendation foundation models. In *SIGIR-AP*, pages 195–204. ACM, 2023.

[15] Wang-Cheng Kang and Julian McAuley. Self-attentive sequential recommendation. In *ICDM*, pages 197–206. IEEE, 2018.

[16] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. Autoregressive image generation using residual quantization. In *CVPR*, pages 11523–11532, 2022.

[17] Lei Li, Yongfeng Zhang, Dugang Liu, and Li Chen. Large language models for generative recommendation: A survey and visionary discussions. *LREC-Coling*, 2024.

[18] Xinhang Li, Chong Chen, Xiangyu Zhao, Yong Zhang, and Chunxiao Xing. E4srec: An elegant effective efficient extensible solution of large language models for sequential recommendation. In *WWW*. ACM, 2024.

[19] Yongqi Li, Nan Yang, Liang Wang, Furu Wei, and Wenjie Li. Generative retrieval for conversational question answering. *Information Processing & Management*, 60(5):103475, 2023.

[20] Yongqi Li, Xinyu Lin, Wenjie Wang, Fuli Feng, Liang Pang, Wenjie Li, Liqiang Nie, Xiangnan He, and Tat-Seng Chua. A survey of generative search and recommendation in the era of large language models. *arXiv preprint arXiv:2404.16924*, 2024.

[21] Jiayi Liao, Sihang Li, Zhengyi Yang, Jiancan Wu, Yancheng Yuan, Xiang Wang, and Xiangnan He. Llara: Aligning large language models with sequential recommenders. In *SIGIR*. ACM, 2024.

[22] Jianghao Lin, Bo Chen, Hangyu Wang, Yunjia Xi, Yanru Qu, Xinyi Dai, Kangning Zhang, Ruiming Tang, Yong Yu, and Weinan Zhang. Clickprompt: Ctr models are strong prompt generators for adapting language models to ctr prediction. In *WWW*. ACM, 2024.

[23] Jianghao Lin, Rong Shan, Chenxu Zhu, Kounianhua Du, Bo Chen, Shigang Quan, Ruiming Tang, Yong Yu, and Weinan Zhang. Rella: Retrieval-enhanced large language models for lifelong sequential behavior comprehension in recommendation. In *WWW*. ACM, 2024.

[24] Xinyu Lin, Wenjie Wang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. Bridging items and language: A transition paradigm for large language model-based recommendation. In *KDD*. ACM, 2024.

[25] Xinyu Lin, Wenjie Wang, Li Yongqi, Shuo Yang, Fuli Feng, Yinwei Wei, and Tat-Seng Chua. Data-efficient fine-tuning for llm-based recommendation. In *SIGIR*. ACM, 2024.

[26] Qijiong Liu, Nuo Chen, Tetsuya Sakai, and Xiao-Ming Wu. Once: Boosting content-based recommendation with both open-and closed-source large language models. In *WSDM*, pages 452–461. ACM, 2024.

[27] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. ICLR, 2019.

[28] Chen Ma, Peng Kang, and Xue Liu. Hierarchical gating networks for sequential recommendation. In *KDD*, pages 825–833. ACM, 2019.

[29] Jianmo Ni, Jiacheng Li, and Julian McAuley. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *EMNLP-IJCNLP*, pages 188–197. ACL, 2019.

[30] Tushar Prakash, Raksha Jalan, Brijraj Singh, and Naoyuki Onoe. Cr-sorec: Bert driven consistency regularization for social recommendation. In *RecSys*, pages 883–889. ACM, 2023.

[31] Zhaopeng Qiu, Xian Wu, Jingyue Gao, and Wei Fan. U-bert: Pre-training user representations for improved recommendation. In *AAAI*, pages 4320–4327. AAAI, 2021.

[32] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q Tran, Jonah Samost, et al. Recommender systems with generative retrieval. In *NeurIPS*. Curran Associates, Inc., 2023.

[33] Xubin Ren, Wei Wei, Lianghao Xia, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. Representation learning with large language models for recommendation. In *WWW*. ACM, 2024.

[34] Yuxin Ren, Qiya Yang, Yichun Wu, Wei Xu, Yalong Wang, and Zhiqiang Zhang. Non-autoregressive generative models for reranking recommendation. *arXiv:2402.06871*, 2024.

[35] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. Bpr: Bayesian personalized ranking from implicit feedback. *UAI*, 2012.

[36] Wentao Shi, Jiawei Chen, Fuli Feng, Jizhi Zhang, Junkang Wu, Chongming Gao, and Xiangnan He. On the theories behind hard negative sampling for recommendation. In *WWW*, pages 812–822, 2023.

[37] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In *CIKM*, pages 1441–1450. ACM, 2019.

[38] Juntao Tan, Shuyuan Xu, Wenyue Hua, Yingqiang Ge, Zelong Li, and Yongfeng Zhang. Towards llm-recsys alignment with textual id learning. In *SIGIR*. ACM, 2024.

[39] Jiaxi Tang and Ke Wang. Personalized top-n sequential recommendation via convolutional sequence embedding. In *WSDM*, pages 565–573. ACM, 2018.

[40] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[41] Aaron Van Den Oord, Oriol Vinyals, et al. Neural discrete representation learning. In *NeurIPS*, volume 30. Curran Associates, Inc., 2017.

[42] Yidan Wang, Zhaochun Ren, Weiwei Sun, Jiyuan Yang, Zhixiang Liang, Xin Chen, Ruobing Xie, Su Yan, Xu Zhang, Pengjie Ren, et al. Enhanced generative recommendation via content and collaboration integration. *arXiv preprint arXiv:2403.18480*, 2024.

[43] Wei Wei, Xubin Ren, Jiabin Tang, Qinyong Wang, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. Llmrec: Large language models with graph augmentation for recommendation. In *WSDM*, pages 806–815. ACM, 2024.

[44] Chuhan Wu, Fangzhao Wu, Tao Qi, and Yongfeng Huang. Userbert: Pre-training user model with contrastive self-supervision. In *SIGIR*, pages 2087–2092. ACM, 2022.

[45] Yunjia Xi, Weiwen Liu, Jianghao Lin, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, Rui Zhang, and Yong Yu. Towards open-world recommendation with knowledge augmentation from large language models. *arXiv preprint arXiv:2306.10933*, 2023.

[46] Junjie Zhang, Ruobing Xie, Yupeng Hou, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. Recommendation as instruction following: A large language model empowered recommendation approach. *arXiv preprint arXiv:2305.07001*, 2023.

[47] Yang Zhang, Fuli Feng, Jizhi Zhang, Keqin Bao, Qifan Wang, and Xiangnan He. Collm: Integrating collaborative embeddings into large language models for recommendation. *arXiv preprint arXiv:2310.19488*, 2023.

[48] Yuhui Zhang, Hao Ding, Zeren Shui, Yifei Ma, James Zou, Anoop Deoras, and Hao Wang. Language models as recommender systems: Evaluations and limitations. In *NeurIPS*. Curran Associates, Inc., 2021.

[49] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, and Ji-Rong Wen. Adapting large language models by integrating collaborative semantics for recommendation. In *ICDE*. IEEE, 2024.

[50] Dixian Zhu, Gang Li, Bokun Wang, Xiaodong Wu, and Tianbao Yang. When auc meets dro: Optimizing partial auc for deep learning with non-convex convergence guarantee. In *ICML*, pages 27548–27573. PMLR, 2022.